

SIT 215
Assignment 1
Problem-Solving
Report

s224326349
Angodavidanelage Rukshan Anthony Dias

Cover Page

Tasks	Completed
Task 1: Environment Creation and problem formation (P/C)	☒
Task 2: Basic navigation implementation (P/C)	☒
Task3: Enhance the environment and heuristics comparison (D)	☒
Task 4: Performance enhancement with an alternative Algorithm (HD)	☒
Task 5: Graphical user interface (Bonus Track)	

Contents

Problem Formation	3
Approach.....	4
Implementation Overview	6
Results and Evaluation.....	9
Conclusion	12
Acknowledgment	13
References.....	13

Problem Formation

We know that disabled wheelchair users face several challenges when it comes to navigation from one place to another in their day-to-day lives compared to what people without disabilities face. These challenges are often ignored when planning routes or making roadways. Some environments still fail to include handicapped services, curbs, and uneven ground. So modern digital mapping tools such as Google Maps or OpenStreet Maps fail to consider the shortest routes for handicapped people, to get from one place to another. If this issue isn't addressed, handicapped people who use wheelchairs could be facing great distress and unexpected physical challenges.

The end goal of this assignment is to use our knowledge and come up with a solution for this problem with a navigation system that is suitable for these handicapped people using a wheelchair. In here I used the A* algorithm and fed real-world surrounding data to show actual infrastructure and geographical circumstances. The algorithm will compare standard and improved heuristics to see the real-world conditions.

For this project, I used the places around my home area which is Vermont South, a suburb of Melbourne located in the state of Victoria. It starts from 1 Dalroy Cres, Vermont South VIC 3133, which mainly is a residential area with access to malls, medical facilities, gyms, and parks close by. The main reason I used this location is because I can relate to the experiences Wheelchair users could be facing based on the road access and terrain features I encounter each day. I realized that this location was perfect for modeling real-world difficulties because it has sidewalks, pedestrian ramps, pharmacies, ATMs, water taps, supermarkets, libraries, and a university district.

To support this, I got the real world coordinates from Google maps and OpenStreetMap for the locations in the area. I basically had to get the latitude and longitude coordinates and include them in the code as shown below.

```
locations = {  
  "Library": (-37.8568, 145.1735),  
  "Shopping Centre": (-37.8572, 145.1750),
```

For handicapped wheelchair users, the routes in this map could be comfortable or not so comfortable. Let's look into some different situations,

- Here, there is a tactical crossing ramp at the "Hospital" node. There
- Though we can see many routes to the "Mall Entrance", those routes lack curb ramps.
- There is only a walking ramp that goes from the "Library" node to the "Public Restrooms".

The above-mentioned locations will be displayed in the below-mentioned screenshot under 'Approach' The end goal of this assignment is to give users a realistic path by simulating these in our graph and map taking the accessible factors into account.

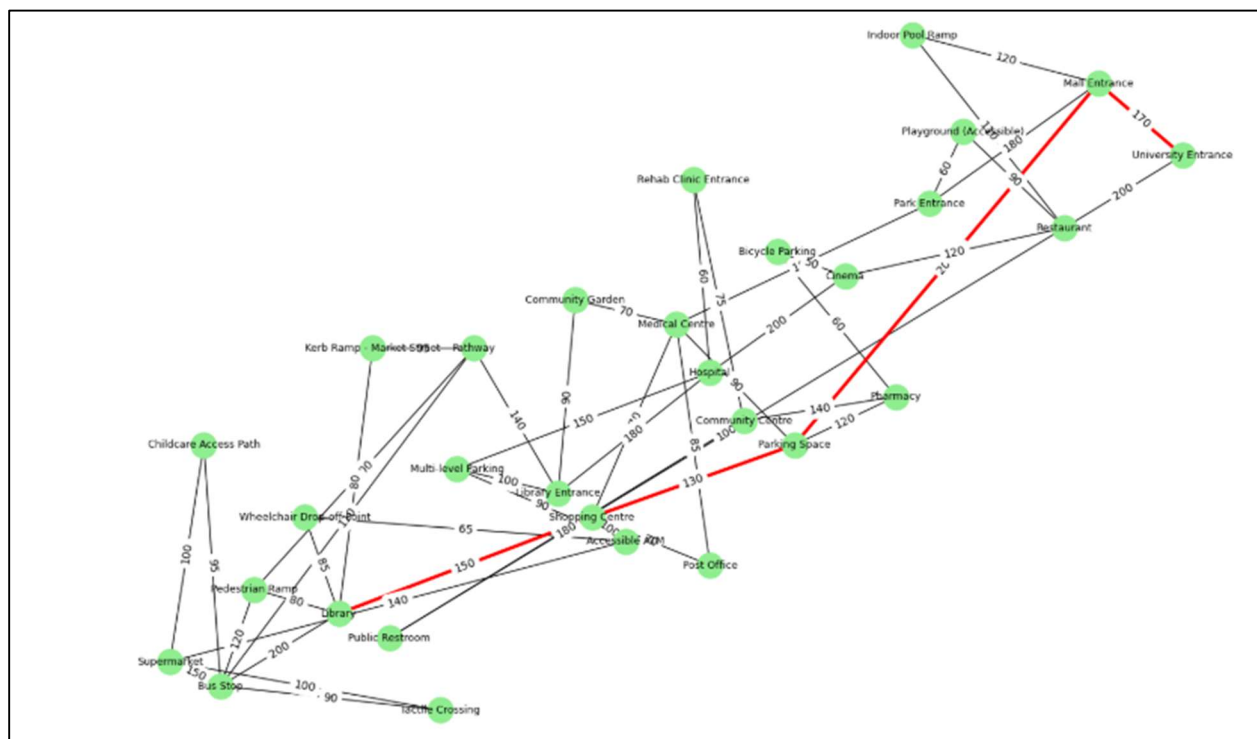
Computationally speaking, the navigation problem was modelled as a graph based search challenge, with accessible connections between locations like ramps, clinics, or shopping malls forming weighted edges. These edges weights takes accessibility issues or actual distances into account. Getting the best routes using search techniques that strike a balance between accessibility and efficiency was the main goal here. By simulating the limitations wheelchair users face, such as missing curb ramps or steep paths,

this problem becomes not just about distance minimization but safe and inclusive path optimization. This approach made it possible to apply computational intelligence tools to everyday accessibility planning in a useful way.

Approach

I used the folium library here to make a .html map for the region I'm planning to do the task on. Then I rounded off the decimal places of the coordinates to 4th decimal place and gathered them as nodes. The edges were defined using a list of tuples, with a starting point and end point. It also displayed the travel cost in meters. This is not the actual walking distance. We use an estimate to conduct the search properly. The distance was calculated using the square root of the sum of the 2 differences of the latitudes and longitudes.

In here I combined ideas from computer intelligence to solve this problem. I used a graph here to show the environment and the cost in meters with nodes sitting in for actual locations and edges for wheelchair accessible paths.



I took initiative to create a map using folium and OpenStreetMap data to give a better understanding of the locations and the path. When we run the ipynb file, this map will be downloaded showing the path by a red marker. It's an html file by the name "wheelchair_accessible_map 30.html". I manually added 30 nodes that match the actual locations. Then used a search algorithm that gives the best path based on lower cost and heuristic. The search strategy used here is the **A* Algorithm using $f(n) = g(n) + h(n)$** .

- $f(n)$ – total estimated cost of the cheapest solution

Implementation Overview

Under this heading, we shall dive into 5 sub-headings to understand the implementation of my task better

1. *Structure of the code and modeling of the graph*

In here the most crucial part is having a well-structured graph. To get the approximate path costs between real-world locations, edges, and their associated weights were stored in a Python dictionary. The nodes/ locations in the graph match important locations on the Vermont South map. It gives us an easily accessible path between them.

```
# Graph with edges and weights
graph = {
    "Library": {"Shopping Centre": 150, "Bus Stop": 200, "Pedestrian Ramp": 80,
    "Kerb Ramp - Market Street": 80, "Wheelchair Drop-off Point": 85},
    "Shopping Centre": {"Library": 150, "Parking Space": 130, "Medical Centre":
170, "Post Office": 70},
    "Parking Space": {"Shopping Centre": 130, "Medical Centre": 90, "Mall
Entrance": 200},
    . . .
}
```

The above code defines a graph that will show the actual portion on the map generated from by the code. By this, the computer doesn't have to search through everything to find the next step since we are using a dictionary, something similar to a lookup table. It can jump straight to where it needs to go. This makes it convenient to check routes and compute trip costs, which makes it easier for navigation.

2. *Heuristic function and A* Agent behavior*

In here I'll be explaining how I utilized the A* algorithm to decide the best path. The algorithm will use a certain calculate to determine which way is the best without just heading to one spot after the other.

```
# A* Heuristic function
def heuristic(node1, node2):
    lat1, lon1 = locations[node1]
    lat2, lon2 = locations[node2]
    return math.sqrt((lat1 - lat2) ** 2 + (lon1 - lon2) ** 2) * 100000
```

so this is something similar to using a simple ruler on a map. By utilizing this function, it will give us the straight line distance between two points. It won't consider any obstacles.

```
# Enhanced heuristic with penalty for non-accessible nodes
def heuristic_enhanced(a, b):
```

```

base = heuristic_euclidean(a, b)
penalty = 0 if "Ramp" in a or "Ramp" in b or "Accessible" in a or
"Accessible" in b else 0.001
return base + penalty

```

This is the addition that was tailor made for Task 3. In here, a small penalty is applied if a certain location does not have keywords like “Ramp” or “Accessible”. What this basically does is, it’s informing the handicapped users that the path could be a bit uneasy. It’s upto the user to decide whether to take another path

3. Node expansion and A* function

We use a priority queue ‘heapq’ is added in the below code to handle open nodes in the execution of the basic search mechanism. The node with / location with the least $f(n)$ is extended at each step.

```

open_set = []
heapq.heappush(open_set, (0, start)) it starts from the first location here
came_from = {} and this tracks the path history

```

After that it initializes the node’s g_score and f_score :

```

g_score = {node: float('inf') for node in locations}
f_score = {node: float('inf') for node in locations}

```

The selected heuristic is used to determine the nodes’s distance to the objective and set its score to 0

The algorithm will keep looping until we have no more locations to check. After that `current = heapq.heappop(open_set)` choose the node with the least cost. Once the final location is checked and confirmed it will then build the path backward using the ‘came_from’ map. the code below was used to get this path

```

while current in came_from:
    path.append(current)
    current = came_from[current]

```

Let’s say that the goal hasn’t been reached yet, then the algorithm will keep exploring all neighbors, `for neighbor, cost in graph.get(current, {}).items():`

Then it computes to see how much it would cost to get to this location from the current `tentative_g_score = g_score[current] + cost`

If the path is better than whatever the algorithm has managed to compute so far, it will record it. By this, we can make sure that the handicapped users will be traveling in the closest direction. The code below is basically used to record the path.

```

g_score[neighbor] = tentative_g_score
f_score[neighbor] = g_score[neighbor] + heuristic(neighbor, goal)
heapq.heappush(open_set, (f_score[neighbor], neighbor))

```

4. *Several Test cases*

We can create a few test cases for example we can use the Library → University Entrance, Post Office → Indoor Pool Ramp, Community Garden → Playground (Accessible), Cinema → Tactile Crossing. We can try these to show the different navigational requirements disabled people could have.

If you change the start and end nodes, make sure to make those changes in where the map is developed, they'll be listed below.

```
optimal_path = ['Library', 'Shopping Centre', 'Parking Space', 'Mall Entrance', 'University Entrance']
```

```
start_node = "Library"
```

```
goal_node = "University Entrance"
```

After the nodes are changed in the code we then run the two versions of A* algorithm as shown below. You can find it in the Task 3 Code cell

```
path_basic = a_star(start, goal, heuristic_euclidean)
path_enhanced = a_star(start, goal, heuristic_enhanced)
```

Here the first line of code will Euclidean distance and the second line of code will use the Enhanced Heuristic which adds on certain penalties.

5. *Graph and Adjacent Matrix Visualization*

In here we basically visualize the graph to get a good representation of the locations and the costs between different locations. I imported the 'NetworkX' library and used it to make a graphical representation. The edges show the accessible routes for wheelchairs and weights based on the distance. Then the locations were connected and this served as a foundation to help present what is happening.

The adjacent matrix shown here is used to simulate the structure of the environment used in a form that the A* Algorithm can comprehend. It keeps data such as,

- The nodes/ locations that are connected (also known as neighbors)
- The cost of traveling between them.

I chose this format because it gives easy and quick access to check the available paths for wheelchair users and efficient computation when checking different neighbors when finding paths. It helps to validate the paths more quickly manually and makes sure all the routes are joined and costed.

In conclusion, the visual representation offered via NetworkX was accurate and helpful even for a non-technical person to understand. It helps us to confirm that every node has been connected and the respective distances are labeled. There is no isolated or missing nodes.

6. Implementation of Dijkstra Algorithm

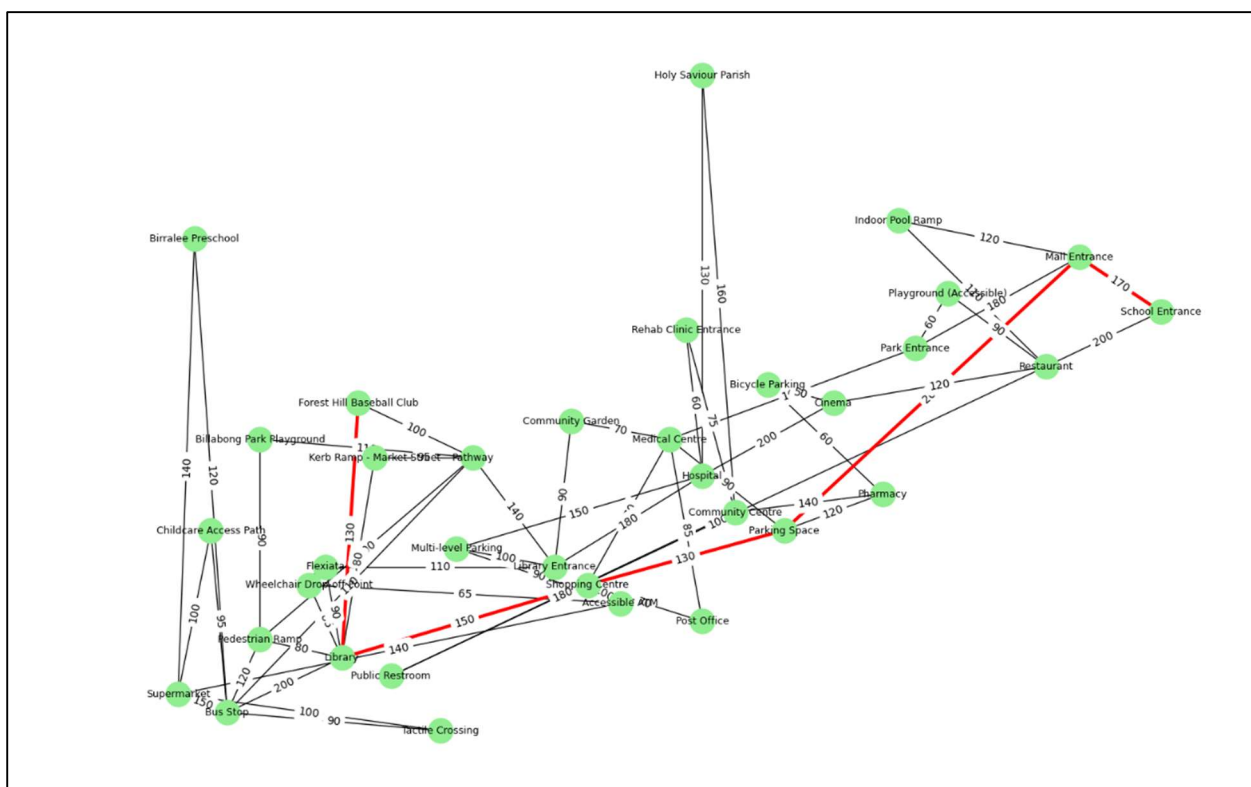
The Dijkstra algorithm is another path finding technique used to complete the HD level task in this assessment. Unlike A*, Dijkstra expands the nodes based on the path costs $g(n)$ which are already known making sure the users get the shortest path without the use of any heuristic functions.

```
def dijkstra(start, goal):
    open_set = []
    heapq.heappush(open_set, (0, start))
    came_from = {}
    g_score = {node: float('inf') for node in locations}
    g_score[start] = 0
```

So in here, the new algorithm is subjected to the same 4 test cases. The cases can be changed in the code using other locations in the map based on your preference.

Results and Evaluation

The project's initial task included creating wheelchair-accessible routes near my home, Vermont South area. Accessible pathways were represented by the edges connecting these nodes and the cost or distance representing the approximate distance in meters were noted on the graph. NetworkX was used to get the graph visualized and the output was clearly showing all the locations connected and with the distances/ cost perfectly assigned.



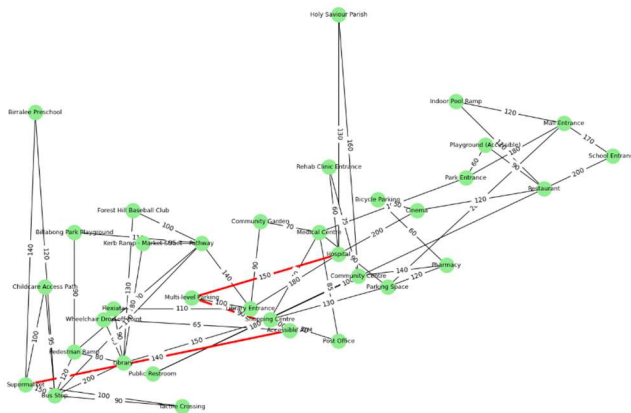
This is the final result and it shows all the 30 nodes/ locations connected with the costs/ distances from point to point.

The A* algorithm was applied on two specific locations to find the best route to the goal. As mentioned above the algorithm used $f(n) = g(n) + h(n)$. As per the calculations from the algorithm, I got the total distance from the selected starting point which is the library, to the School entrance was 650m. We can use change the locations by copying the location names mentioned in the initial stage of the code to find the paths and the costs to different locations. I've have used 3 test scenarios and output them in the code to show that the algorithm works perfectly showing the costs for each as well. The connectivity of the nodes and edge weights were properly displayed in the NetworkX result visualization.

A* Result Summary

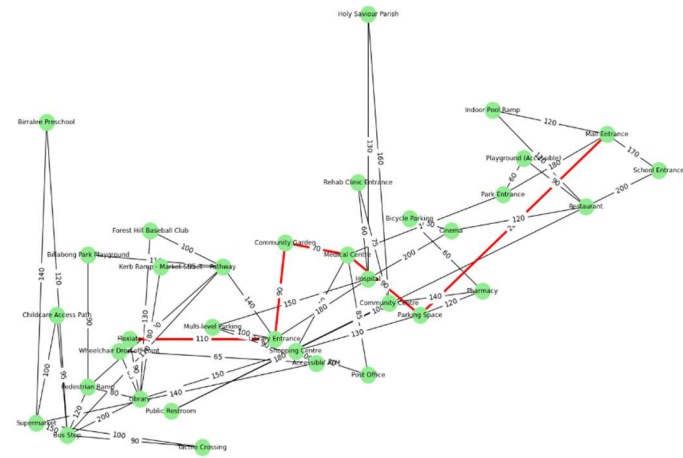
Shortest path from Supermarket to Hospital:
Supermarket → Accessible ATM → Multi-level Parking → Hospital
Total Distance: 380 meters

A* Path from Supermarket to Hospital with Costs



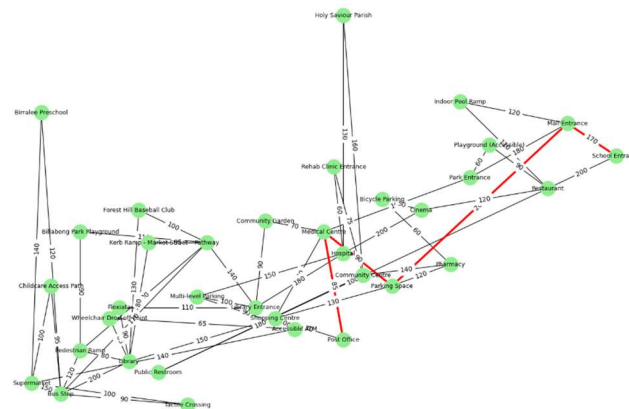
Shortest path from Flexiata to Hall Entrance:
Flexiata → Library Entrance → Community Garden → Medical Centre → Parking Space → Hall Entrance
Total Distance: 560 meters

A* Path from Flexiata to Hall Entrance with Costs



Shortest path from Post Office to School Entrance:
Post Office → Medical Centre → Parking Space → Hall Entrance → School Entrance
Total Distance: 545 meters

A* Path from Post Office to School Entrance with Costs



In the final section, we will dive deep into the implementation of the A* algorithm using both a basic and enhanced heuristic. It will use several realistic test scenarios to check the usability and dependability of the pathway. As mentioned in the code under task 3, four different scenarios have been used to check the effectiveness of the enhanced environment and heuristic.

	Start	Goal	Basic Cost	Enhanced Cost
0	Library	University Entrance	650	650
1	Post Office	Indoor Pool Ramp	495	495
2	Community Garden	Playground (Accessible)	290	290
3	Cinema	Tactile Crossing	680	680

Based on the above findings, we can see that whenever the heuristic was applied, the path stayed the same. The tests we did above not only calculated the overall path costs but also it helped in examining the ways in which the heuristics affected the decision making process. The Vermont South region has a decent amount of resident aged 65 and above. This is good news because if we are getting an output like this, it means the accessible path segments are already present in the area. Therefore, it reduces the need to change the routes using improved heuristics. Therefore the 0.001 penalty we added didn't have any effect on the final path since they were already accessible. This shows that with a good urban layout the need to have a complicated algorithmic logic is less.

Furthermore, as mentioned in the D task, the Dijkstra Algorithm was put into practice and compared with A* in the same environment using the test cases used above. As a result, we got the same values as the above table. This proves the accuracy of the A* algorithm was demonstrated because in each test situation, the optimal pathways and related costs generated by both algorithms Dijkstra and A* were similar.

What we got out of here,

- Use of A*'s heuristic enabled examine fewer nodes in complex segments which makes it more efficient when calculating.
- Dijkstra on the other hand considered all the paths equally which might not be as effective in larger settings.

	Start	Goal	A* cost	Dijkstra cost
0	Library	University Entrance	650	650
1	Post Office	Indoor Pool Ramp	495	495

2	Community Garden	Playground (Accessible)	290	290
3	Cinema	Tactile Crossing	680	680

(Refer the table generated in the ipynb file as it outputs a better table)

The test results showed that A* allowed it to get the same outcomes as Dijkstra with less calculation steps. This is very important to handicapped wheelchair users when navigating. Dijkstra's lack of heuristic could be the reason for slower calculations in larger complex settings even though it has equal accuracy. This confirms all the locations were connected well.

Accuracy and safety are more crucial in wheelchair navigation in the real world than speed. Therefore, Dijkstra's thorough method guarantees accuracy, which is essential when users have to navigate clear of inaccessible paths. But large dynamic maps can of course benefit from A*'s large scalability and adaptability to accessible heuristics.

Conclusion

The assignment has shown me how well computational intelligence (CI) algorithms, especially A*, can be utilized to solve problems in navigation and our day-to-day lives. In order to create how a handicapped wheelchair user may navigate to common public places like Libraries, Shopping centers, transportation clinics, and parks, the task was focused on using the environment around a location. For this I chose the location I live in which is Vermont South. The environment was used to reflect on the accessibility situations.

Using the A* algorithm was a success and this showed that CI tools can help choose the perfect path in a complex environment using both the basic and enhanced heuristic. The basic heuristic chose the path under normal conditions without any penalties, but the enhanced heuristic version used penalties to avoid uncomfortable roads for handicapped wheelchair users to use.

The way the environmental rules affected the search for a better route is what made this assignment unique. There were restrictions that were added into location/ nodes in a small manner such as 'Ramp' and 'Accessible' without directly labeling them as unusable check points or as an obstruction. By using the small penalty of 0.001, the search agent was encouraged to look for an easy going route for the wheelchair user. This is very similar to real world planning where the goal of urban planning is to govern people by carefully planning the routes for everyone including wheelchair people to use without restricting routes often unless it's really necessary.

Using Dijkstra algorithm helped us in comparing A* and it showed the superior performance of A* for quick and accessible path finding using navigation systems for wheelchair users. By this we can say that, when finding navigational solutions, the difference supports the use of heuristic prioritized search algorithms.

Furthermore, an important factor in this was seeing the consistency of the basic and the enhanced heuristic. It showed that the recommended routes in Vermont South were effective and

usable and that they were already optimized for wheelchair / handicapped users. This confirms the accuracy of both the algorithm and the environment.

To sum up this study, it demonstrated how CI techniques like A* could be used to solve problems people face in their everyday lives. Likewise, we can create systems that are not only useful but also aware of the requirements of various users by fusing intelligent algorithms with mindful planned surroundings. Who knows what the future holds, new navigation models could be integrated with adaptive mobility planning systems, real time sensors input, and terrain aware navigation.

In my opinion, I believe this assignment shows how computer intelligence can improve everyday navigation and make it safe, and smarter for everyone, especially handicapped and wheelchair users.

Acknowledgment

Without the assistance from the Deakin University Module lecturers this project wouldn't have been feasible. The direction and structure of the implementation were guided by their teachings during the SIT215 practical sessions.

Furthermore, a special thanks should be extended to my fellow SIT215 classmates who engaged in cooperative conversations and ideas during the practical sessions and out of class hours offering insightful view points and assistance with solving problems I faced throughout the development process. The worldwide mapping community's contribution and OpenStreetMap which made it easy to create a landscape of the Vermont South region with accuracy and realism.

References

1. *Sign in - Google Accounts* (no date b). https://www.google.com/maps/d/u/0/edit?sjid=2186595426190718294-NC&mid=1O1ttRm5RU_YN3n89fD6vDEIvPqIui78&ll=-37.85754972965856%2C145.17407027108914&z=18.
2. *OpenStreetMap* (no date). <https://www.openstreetmap.org/#map=5/-28.15/133.28>.
3. Deakin University (no date) *DeakinSync*. <https://sync.deakin.edu.au/home>.
4. *Google Maps* (no date). https://www.google.com/maps/@-37.8547878,145.1706999,15z?entry=tту&g_ep=EgoyMDI1MDQwOC4wIKXMDSOASAFQAw%3D%3D.